



Caruca v2

LLM-based specification mining for opaque shell commands

Data & Telemetry Schema

What gets recorded, and how results are stored

SOURCE `ai_docs/prep/data_telemetry_schema.md`

GENERATED `August 29, 2026 at 7:08 PM`

PAGES `6`

CARUCA V2 — RESEARCH PROJECT

Table of contents

Data & Telemetry Schema

Reused from v1 (unchanged)

Telemetry Record

Comparison-Result & Percent-Change Roll-Up

Sandbox Execution Result

Storage Approach

Open Items Carried From the Review

Data & Telemetry Schema

Produced via the `08_generate_initial_data_models` planning pass, corrected per the audit in `~/claude/plans/review-our-progress-so-delightful-token.md` (2026-08-29) before being saved — see that plan for the full reasoning behind the corrections noted inline below.

Reused from v1 (unchanged)

`FSInteraction`, `Trace`, `CommandInvocationTraces`, `CommandInvocationTraceSet`, `Traces` (the `RootModel[list[CommandInvocationTraceSet]]` JSON boundary) — all in `tracer/data.py`, confirmed via direct read, not assumed. None of these have any telemetry, comparison, or reproducibility-metadata concept today; the three schemas below are genuinely new, not extensions of something v1 already has.

One structural note carried forward from that read: v1's annotation logic isn't a separate reader over generic trace JSON — it's baked directly into

`CommandInvocationTraces.to_annotation()` (hardcoded `.jpg` suffix stripping, a `stdin_1 / _2` naming convention for splittability, no symlink/permission-denied `FSInteraction` variants). Extending it for the v2-extended profile means touching that method, not just accepting richer input.

Telemetry Record

Every LLM call (v1's syntax-spec step, the naive-LLM baseline's plain and augmented conditions, and the tool-augmentation layer's per-tool calls) emits one record:

- `run_id`, `command`, `component` (`baseline` | `naive_llm` | `tool_augmentation`), `condition` (`plain` | `augmented` — applies to naive-LLM's two labeled conditions), `model_id`, `prompt_tokens`, `completion_tokens`, `cost_usd`, `wall_clock_seconds`, `seed`, `prompt_hash`, `timestamp`
- Stored as a **sidecar JSON file** per run (e.g. `outputs/CMD.telemetry.json`), joinable to `Traces` or the syntax-spec `.py` output by `run_id` — never embedded in a v1-compatible file, since those are read directly by v1's own tooling and telemetry fields would pollute that contract.
- `retry_count` is deferred until a real retry loop exists — the naive-LLM baseline deliberately has none (Eiers' "don't min-max" guidance).
- Implementation note: `dspy.settings.configure(lm=lm)` (`llm.py:115`) rebinds the LM **process-wide** — a concurrency hazard if Baseline Instrumentation ever runs multiple

telemetry-capturing calls (different models/seeds) concurrently. Serialize those calls, or isolate per-process.

Comparison-Result & Percent-Change Roll-Up

One record per command, per dimension, per comparison condition:

- `run_id`, `command`, `dimension`, `comparison_system` (`naive_llm_plain` | `naive_llm_augmented`), `profile` (`v1_faithful` | `v2_extended` | `n/a`), `method`, `v1_value`, `v2_value`, `match_rate`, `exact_match`, `percent_change`, `n_samples`, `evidence_refs`

Correctness-collapse method (corrected from the initial draft): `cmp_specs.py`’s own `correct_percentage` (line 152) divides a mismatched-**element** count by a **dataclass-field** count (`count_syntax_elements`, line 135, sums fields despite its name) — confirmed via direct code read, these are different units, so it is not a true `matched/total` argument-level fraction. `exact_match` (`rate == 1.0` iff `diff_count == 0`) is unaffected and safe to compute from v1’s script as-is; `match_rate` must be computed by a **v2-side corrected** element-count before it’s trustworthy as an argument-level percentage comparable to the paper’s own 99.7% (§2.2) figure.

method **distinguishes which correctness methodology produced a value** — these are NOT interchangeable and must never be blended without this tag (confirmed via paper audit, §7.1 vs §7.2 use genuinely different methodologies):

- `q2_syntax_diff` — argument-by-argument syntax-spec diff (Tier 0, `cmp_specs.py`-style, ground truth verified as genuinely hand-curated via git history on `syntax_specs/*.py`)
- `annotation_diff` — post-annotation diff against `benchmarks/annotations/` (Tier 1, what this project designed in the component-functionality pass; a real check, but NOT the paper’s Q1 methodology)
- `q1_execution` — real PaSh/ShellCheck/Shseer suite reruns against generated output (Tier 1, matches the paper’s actual Q1 methodology: PaSh’s benchmark suite with output-hash comparison, ShellCheck’s 2.2K-test suite, Shseer’s 12 bug-scripts — not implemented until Tier 1)

`profile` is explicit `n/a` (not omitted) for dimensions that don’t route through the sandbox — cost, and Q2-style syntax correctness, both need no tracing.

Sandbox Execution Result

Extends `Traces`, doesn't replace it — the isolation backend (Docker/overlayfs extension vs. Firecracker vs. gVisor) is still an open choice, and the result must stay `Traces`-shaped regardless of which one is picked, since `strace/ptrace` visibility is a hard constraint and the Evaluation Harness shouldn't need a different reader per backend.

Two additions are backend-independent and committed now, since they're data-model gaps, not backend questions:

- `FSInteraction` gains new variants for the v2-extended profile: symlink creation/traversal, permission-denied access, SIGPIPE-terminated write.
- `CommandInvocationTraces` gains a `profile: v1_faithful | v2_extended` field (so a trace record self-identifies which fixture profile produced it — this is what populates the comparison schema's `profile` field above). `CommandConfig` gains an `env_vars: dict[str, str]` field — environment-variable variation (paper §2.2's own disclosed limitation, `evaluation_gaps.md` gap #14) is a precondition on the invocation, not a filesystem interaction, so it hooks in at the config level, not `FSInteraction`.

Left as a placeholder: `backend_metadata: dict`, for whatever backend-specific fields the eventual choice needs (e.g. a Firecracker VM ID). Also left as implementation work, not a schema decision: how `__readwrite`'s path-classification logic should resolve symlink targets or classify permission-denied traces as read/write/neither — that depends on how the chosen backend reports paths, not on the schema.

Storage Approach

Hybrid, not a single pick from the two options originally posed:

- **JSON sidecar files** (extending v1's own `outputs/CMD.json` convention) are the source of truth for telemetry and comparison-result records — trivially diffable and inspectable while these schemas are still under active change, and something a paper reviewer can be handed directly.
- **A derived SQLite layer**, built by loading those JSON files, exists specifically for the Evaluation Harness's aggregation and percent-change roll-up queries across the full command set. It's a queryable cache rebuilt from the JSON records, not a second source of truth — no migration story needed, since it can always be regenerated.

Open Items Carried From the Review

- The `cmp_specs.py` denominator fix (element-count, not field-count) is a concrete implementation task for whenever the Evaluation Harness is actually built — this pass only accounts for it in `method`’s value space, it doesn’t implement it.
- Ground-truth provenance for `syntax_specs/*.py` was verified via git history (92 commits, 4 human authors, explicit “ground truth”/“gt” commit-message language, a “with types” pass matching the paper’s own reported type-misclassification discrepancy) — resolved, no schema impact, not re-litigated here.